Software Requirements Specification (SRS) - Fintech360

1. Introduction

1.1 Purpose

This document defines the software architecture, data flow, and processing requirements for the Fintech360 scalable data pipeline.

1.2 Scope

This pipeline captures simulated fintech data generated via Python Faker, stores it in a transactional database (AWS RDS), extracts it to a data lake (AWS S3), and processes it through a Medallion architecture in Databricks. The final output serves Data Analysts, Machine Learning models (for credit risk and underwriting), and Management dashboards (Power BI).

2. System Architecture & Data Flow

2.1 Data Generation & Ingestion

- **Mock Data Generation:** A Python `faker` script generates realistic fintech entities (customer profiles, loan applications, KYC status, repayment transactions, DPD tracking).
- **Transactional Source:** The generated records are ingested into an AWS RDS (PostgreSQL) database, simulating a live loan origination and core banking system.

2.2 Data Lake Landing

- **S3 Landing Zone:** Raw data is extracted from AWS RDS (via micro-batches or CDC) and lands in an AWS S3 bucket as flat files, decoupling the transactional database from analytics compute.

2.3 Databricks Processing (Medallion Architecture)

- **Bronze Layer (Raw):** Ingests raw data from S3. Retains an immutable historical log of all transactions and state changes.

- **Schema Validation Gate:** Enforces strict checks on incoming data before entering the Silver layer. Verifies columns, data types, strict nullability rules, and manages safe schema evolution.
- **Silver Layer (Cleaned & Standardized):** Cleanses, deduplicates, and formats data. PII (Personally Identifiable Information) is masked or tokenized at this stage.
- **Data Quality Gate:**
    - **Null checks:** Ensures critical identifiers (e.g., `loan_account_id`) are present.
    - **Duplicate checks:** Prevents double-counting of payment transactions.
    - **Range checks:** Validates business logic constraints (e.g., Days Past Due is positive, interest rates are within legal limits).
    - **Referential integrity:** Ensures every repayment maps to a valid customer and loan account.
- **Quarantine (Dead Letter/Fail Route):** Records failing the quality gate are routed to an Error Monitoring quarantine table. This prevents bad data from corrupting downstream metrics.
- **Gold Layer (Business Transformation):** Highly aggregated, denormalized tables (e.g., portfolio quality, collection efficiency, acquisition cost vs. risk). Governed by Unity Catalog for access control.

3. Downstream Consumers

3.1 Data Analysts

- **Access:** Direct querying of the Gold layer via Databricks SQL.
- **Use Cases:** Ad-hoc EDA, vintage analysis, roll-rate tracking, and identifying root causes for customer leakage or default spikes.

3.2 Machine Learning Models

- **Access:** Consume engineered features primarily from the Silver and Gold layers.
- **Use Cases:** Predictive modeling for credit risk decisioning, assigning internal risk deciles, expected loss calculation, and automated loan approval workflows.

3.3 Management

- **Access:** Power BI dashboards connected to the Unity Catalog / Gold layer.
- **Use Cases:** Tracking high-level KPIs balancing growth, risk, and profitability (e.g., Portfolio at Risk (PAR), Non-Performing Assets (NPA), Cost of Acquisition, and overall conversion funnels). Upstream aggregations allow Power BI to utilize query folding for sub-second dashboard rendering.

4. Non-Functional Requirements

- **Scalability:** The pipeline must leverage distributed PySpark compute to handle variable data volumes natively without bottlenecking.
- **Reliability:** The explicit pass/fail data quality routing guarantees data trust.
- **Security:** Row-Level Security (RLS) and Column-Level Security (CLS) enforced via Unity Catalog to isolate sensitive financial data.